

# Chou Ting-Rui (Carson)

Python Automation & Data Pipeline Engineer

Taiwan · Remote · moneycometomywallet@gmail.com

---

I build systems that run themselves — data comes in, gets processed, ships out, and pages me when something breaks. My focus is **correctness under silent failure**: data pipelines, payment webhooks and scheduled jobs, where the main risk is not a crash but a wrong number that nobody notices for three months.

## Failure modes I look for

- A `.get(key, 0)` that returns None because the key exists with a null value.
- A validation gate that quietly lets bad data through because someone made it “more tolerant”.
- A timeout heuristic that kills healthy long-running jobs because it checks file mtime and never checks liveness.

## Selected work (own production systems)

### Video Rendering Pipeline — 13× faster, zero hardware cost

Rendering took 294 seconds per clip. The obvious fix was a faster GPU. I measured first, and the measurement contradicted the assumption: CPU time across the entire render was about 0.2 seconds — the processor sat idle for five minutes. The bottleneck was I/O, not compute: frames were being piped one at a time into the encoder over stdin. I rewrote that layer as a pure ffmpeg architecture (concat demuxer, single filtergraph, one encode pass). **294s → 22.5s on identical hardware**. The previous implementation is retained as an automatic fallback — I do not ship a single point of failure.

### Taiwan Equity Market Data Service

Consolidates Taiwanese stock market data from several official sources into one queryable system across eight independent domains — institutional flows, margin trading, shareholder distribution, fundamentals, valuation, quotes, news, technical indicators. Every external parser has its own tests: government sites redesign their pages regularly, and tests turn a silent data corruption into a loud failure on day one. Includes health checks and an MCP query interface.

### Payment Webhook & Subscription Delivery Backend

Signature verification, multi-source event normalisation, transaction ledger, product delivery, subscriber management. Two design decisions worth noting: unknown plan codes still deliver — if a paying customer hits a tier the system has never seen they get their product, and we investigate afterwards; and pricing has exactly one source of truth, enforced by a test, so marketing copy and charged amounts cannot drift apart.

## Testing

**431 automated tests currently passing** across my data and webhook layers. Every external-source parser is covered, because upstream sites change their HTML without warning — with tests you find out the day it breaks, not three months later when you notice the data was wrong the whole time. I also design fail-closed: when the system cannot determine something it refuses rather than guessing. Errors should be loud, not silent.

## Skills

Python · pandas · NumPy · ETL & data pipelines · Web scraping · Playwright · FFmpeg · REST APIs · Webhooks · Stripe · pytest · Task scheduling · SQL · Financial data · Backtesting · Windows/Linux automation